
Does Simulation Feedback Improve LLM-Generated RTL?

Arnav Gaddam

University of Illinois Urbana Champaign
agaddam2@illinois.edu

Prineet Parhar

University of Illinois Urbana Champaign
parhar2@illinois.edu

Abstract

In this project we test a simple question about HDL code that is generated through LLMs. If the first answer given back by the LLM fails simulation, does giving the model the error output and asking it to repair the design actually help? We compare a one-shot baseline against a repair harness where the simulation results are fed back into the model on a 15 task subset of the CVDP RTL code generation tasks. Across three baseline runs and three repair runs, the baseline passed 22/45 task trials, while simulation repair passed 33/45, marking an improvement of the mean pass rate from 48.9% to 73.3%. The improvement was mainly in compile failure recovery and in description-to-RTL tasks, while code completion tasks remained substantially harder.

1 Question

The question we are trying to answer is *"Does simulation-feedback repair improve correctness compared with single-shot RTL generation on a small set of standardized problems?"*

This question came from a pretty practical place. When an LLM writes software, running tests and feeding back the traceback can sometimes help it fix its own mistake. We saw this earlier through working on homework 5. We wanted to see whether the same idea works for RTL. This is not obvious, because Verilog can compile while still describing the wrong circuit, and some failures only show up after several cycles of simulation. In hardware, a plausible looking module is not enough. The module has to preserve the required interface, compile under normal tools, and pass the actual functional and simulation level tests.

Prior work like VerilogEval and RTLLM uses simulation-based evaluation for generated RTL [Liu et al., 2023, Lu et al., 2023]. CVDP pushes this idea into a broader set of hardware design and verification problems [Pinckney et al., 2025]. We use CVDP because it gives us real task prompts and real open-source harnesses, but we narrow the project to one comparison: no feedback versus feedback after failure.

2 Hypothesis/Predictions

Before running the final experiment, we wrote down our prediction in `predictions.md` so that we would not quietly move the goalposts after seeing results. Our hypothesis is that the repair harness should pass more tasks than the single-shot baseline.

Our predicted range is:

- Single-shot baseline: about 25–45% of tasks pass.
- Simulation repair with one repair attempt: about 40–60% of tasks pass.

We expect the repair loop to help most when the first answer is close but has a fixable mistake. For example, a wrong port name, a syntax issue, a missing reset branch, or a small local logic bug that the verifier log points to clearly are some of the prime suspects we seek to address here. We expect it to help less when the model misunderstands the whole state machine or when the simulation output only says that an assertion failed without making the real design error obvious.

3 Problem Selection

The task suite comes from NVIDIA’s public CVDp paper. We selected 15 problems. Eight of them are description-to-RTL problems and seven of them are code completion problems. Each selected problem asks for one RTL output file, does not require extra input files, and includes the original CVDp simulation harness.

The selected rows are stored in `datasets/cvdp/cvdp_v1_1_0_nonagentic_no_commercial_subset.jsonl`. The source URL, task IDs, and selection rules are recorded in `datasets/cvdp/cvdp_subset_manifest.json`.

Task type	Tasks	Notes
Description-to-RTL	8	Full module from natural language (NL) specification
Code completion	7	Prompt contained partial code completion
Total	15	Easy, non-agentic, non-commercial split

Table 1: Selected CVDp task suite. We use only rows that fit the same single file generation format and do not require external context files.

4 Validation

We distinguish execution success from domain success because this distinction is easy to blur in code generation projects.

Execution success means the experiment infrastructure ran correctly. For example, the candidate RTL was written to the expected file, Docker started, the CVDp harness ran, and the result was recorded. If Docker is missing or the image cannot be found, that is an experiment setup failure, not evidence that the model cannot design RTL.

Domain success means the generated RTL actually passed the hardware tests. A test only counts as passing if it satisfies all of these checks:

1. The generated RTL is written to the single output path requested by the CVDp row.
2. The candidate compiles under the original CVDp open-source simulation harness.
3. The compiled design passes the CVDp cocotb/pytest tests for that task.

Each trial writes a structured result with the pass flag, failure reason, compiler log, simulation log, attempt number, and repair metadata. That way, our final table can be traced back to the JSON logs for further inspection.

5 Harness Comparison

We compare two harnesses while keeping the task suite, model provider, temperature, timeout, extraction logic, Docker image, CVDp files, and verifier fixed.

Single-shot baseline. The baseline prompts the model once. The harness extracts the Verilog candidate, checks that the requested module is present, writes it into the CVDp task directory, and runs the verifier. There are no repair attempts.

Simulation-feedback repair. The repair harness starts with the exact same first prompt. If the candidate fails the interface check, compilation, or simulation, the harness sends the previous candidate plus concise verifier feedback back to the same model.

The baseline config is `configs/final_baseline.yaml`. The repair config is `configs/final_sim_repair.yaml`. Both use `dataset_format: "cvdp"`, `temperature: 0.2`, and a Docker backed CVDP verifier.

6 Results

We ran three independent baseline runs and three independent repair runs. Each run used the same problems, CVDP Docker verifier, and OpenRouter model. Table 2 summarizes pass rate across runs. Figure 1 shows the individual variation.

Harness	Runs	Tasks/run	Passed/run	Pass rate
Single-shot	3	15	7.33 ± 0.58	$48.9\% \pm 3.8\%$
Simulation repair	3	15	11.00 ± 1.00	$73.3\% \pm 6.7\%$

Table 2: Main harness comparison over three runs per condition. The $\pm$ value is the standard deviation across runs.

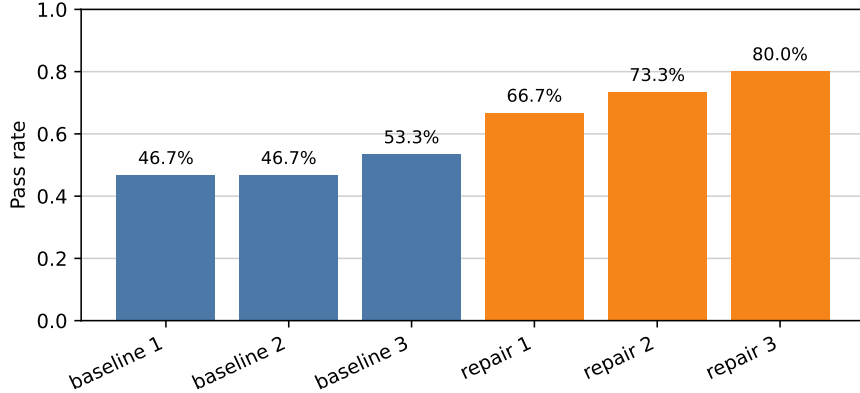


Figure 1: Pass rate for each saved run. Repair runs are consistently above the baseline runs.

The baseline passed 22/45 total task trials, while simulation repair passed 33/45. The baseline averaged 7.33 passing tasks per run and repair averaged 11.00. This is a 24.4 percentage improvement in average pass rate. The observed baseline rate, 48.9%, was slightly above our predicted 25–45% range, as was our repair rate, 73.3%, above our predicted 40–60% range. So the direction of the prediction was correct, but both harnesses, especially repair, performed better than expected.

Harness	Task style	Passed/trials	Pass rate
Single-shot	Code completion	6/21	28.6%
Single-shot	Description-to-RTL	16/24	66.7%
Simulation repair	Code completion	9/21	42.9%
Simulation repair	Description-to-RTL	24/24	100.0%

Table 3: Pass rate split by CVDP task style. NL-to-RTL tasks benefited more from one repair attempt than code completion tasks.

The strongest result is in the description-to-RTL subset. Baseline passed 16/24 description-to-RTL trials, while repair passed 24/24. Code-completion tasks were harder: baseline passed 6/21 and repair passed 9/21. This difference is important for interpreting the result. The repair loop appears effective when the model can use concise compiler or assertion feedback to patch a self-contained generated module. It is less effective when the task prompt includes partial-code constraints that the model must preserve and complete precisely.

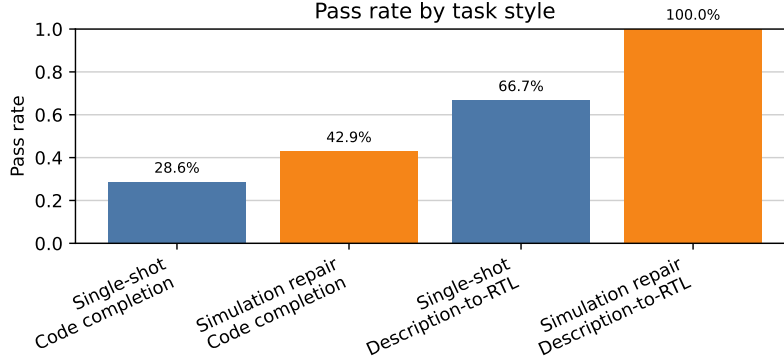


Figure 2: Problem pass rates aggregated over all runs. The gap between problem styles suggests that prompt contained code completion is a different failure regime than code generation.

7 Failure Analysis and Honest Discussion

We classified failures using the structured verifier result recorded in each `results.jsonl`. Table 4 aggregates failures across all trials.

Harness	Failure reason	Count
Single-shot	compile_failed	20
Single-shot	simulation_failed	3
Simulation repair	compile_failed	11
Simulation repair	simulation_failed	1

Table 4: Failure reasons across all failed trials. Counts are over 45 task trials per harness.

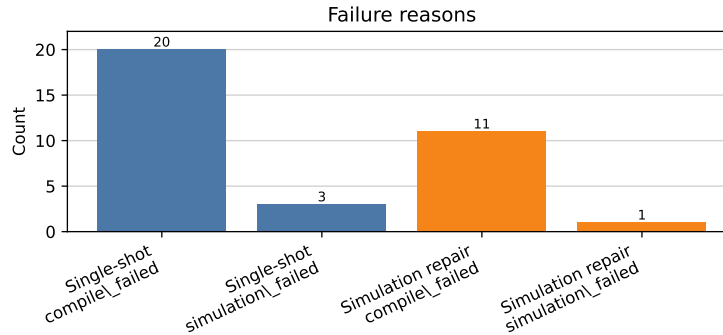


Figure 3: Failure reasons aggregated over all trials. Repair primarily reduces compile failures, which is consistent with the repair prompt receiving direct compiler diagnostics.

The main area of improvement was compile error recovery. The baseline produced 20 compile failures across 45 trials, while the repair runs produced only 11. Simulation failures also decreased, from 3 to 1, but there were too few behavioral failures to make a strong claim about deeper debugging. In the repair runs, several tasks passed only after the feedback attempt. For example, the priority encoder and binary-to-one-hot decoder repeatedly failed initially because the generated Verilog assigned to outputs declared as wires. After we surfaced the actual Icarus diagnostic in the repair prompt, the model was often able to fix the declaration style.

Simulation feedback might improve the harness score without making the model “understand” the design much better. If the log points to a simple syntax or naming issue, the model can often patch it.

Run	First attempt	Fixed by feedback	Still failing	Tasks
repair_1	7	3	5	15
repair_2	9	2	4	15
repair_3	7	5	3	15

Table 5: Repair run outcomes split into problems that passed on the first attempt, problems fixed by the feedback attempt, and problems still failing after repair.

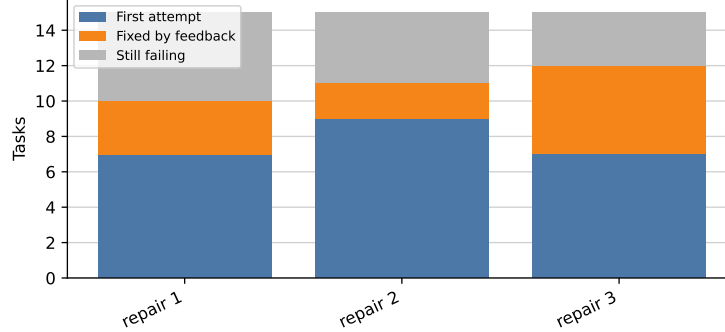


Figure 4: Repair outcome by run. Ten of the 33 repair successes occurred only after feedback, so the improvement is not only due to lucky first attempts in repair runs.

If the failure is a deeper, behavior level problem, the same feedback may not be enough. This is also model dependent too. A stronger frontier model may actually have these capabilities that can only be tested and brought out through testing on the most difficult benchmark problems. We especially expect sequential tasks and FSM design tasks to be harder because the bug may depend on inputs rather than one obvious expression.

Another issue is overfitting to tests. Passing the CVDP harness is stronger than just compiling, but it is still not a formal proof that the RTL is correct for all possible inputs.

8 Reproducibility

The repository includes the selected problems, configs, harness code, and analysis scripts. The user should plugin their own API keys to reproduce our results.

The expected reproduction flow is:

```
python3 scripts/select_cvd_subset.py \
/path/to/cvdp_v1.1.0_nonagent_code_generation_no_commercial.jsonl

export CVDP_SIM_IMAGE=nvidia/cvdp-sim:v1.0.0
PYTHONPATH=src python3 scripts/run_experiment.py configs/final_baseline.yaml
PYTHONPATH=src python3 scripts/run_experiment.py configs/final_sim_repair.yaml
PYTHONPATH=src python3 scripts/analyze_results.py runs/*/results.jsonl
make
```

The Docker image is built from the official CVDP repository using its simulation Dockerfile. The final configs use OpenRouter, so a reproducer should also use the exact model name in `OPENROUTER_MODEL`.

9 Limitations and Future Improvements

We only have 15 problems, so the results still have high variance even with three runs per harness. We also tested standalone RTL modules, not large hardware projects with many files. Also, the repair

harness got extra model calls, so the repair harness method is more expensive than the baseline. Finally, the result depends on the exact model, prompt, temperature, Docker image, and CVDP harness behavior.

The biggest future improvement would be to run multiple models and report confidence intervals instead of one pass-rate table. We would also like to add a cost column, separate compile-fix improvements from true behavioral improvements, and include a few hand written case studies showing when repair helped or failed. A more ambitious version would combine simulation feedback with formal checks, but that would be a larger project than this one.

A Run-Level Results

Table 6 gives the per-run result used to construct Table 2.

Run	Harness	Tasks	Compiled	Simulated	Passed	Failures
baseline_1	Single-shot	15	8	7	7	compile_failed: 7, simulation_failed: 1
baseline_2	Single-shot	15	8	7	7	compile_failed: 7, simulation_failed: 1
baseline_3	Single-shot	15	9	8	8	compile_failed: 6, simulation_failed: 1
repair_1	Simulation repair	15	10	10	10	compile_failed: 5
repair_2	Simulation repair	15	12	11	11	compile_failed: 3, simulation_failed: 1
repair_3	Simulation repair	15	12	12	12	compile_failed: 3

Table 6: Per-run result summary.

B Per-Task Results

Table 7 shows per-task pass counts across the three baseline runs and three repair runs. The final column counts repair successes that occurred on the second attempt, after verifier feedback was supplied.

Task	Style	Base pass	Base trials	Repair pass	Repair trials	Fixed
cvdp_copilot_64b66b_decoder_0001	Code completion	3	3	3	3	0
cvdp_copilot_8x3_priority_encoder_0001	Description-to-RTL	0	3	3	3	3
cvdp_copilot_Attenuator_0001	Code completion	0	3	0	3	0
cvdp_copilot_Carry_Lookahead_Adder_0001	Description-to-RTL	3	3	3	3	0
cvdp_copilot_barrel_shifter_0001	Description-to-RTL	2	3	3	3	0
cvdp_copilot_bcd_adder_0001	Code completion	0	3	1	3	1
cvdp_copilot_bcd_counter_0001	Description-to-RTL	3	3	3	3	0
cvdp_copilot_bcd_to_excess_3_0001	Description-to-RTL	2	3	3	3	3
cvdp_copilot_binary_to_BCD_0001	Code completion	2	3	2	3	0
cvdp_copilot_binary_to_gray_0001	Code completion	0	3	0	3	0
cvdp_copilot_binary_to_one_hot_decoder_0001	Description-to-RTL	0	3	3	3	2
cvdp_copilot_edge_detector_0001	Description-to-RTL	3	3	3	3	0
cvdp_copilot_flop_0001	Code completion	1	3	3	3	1
cvdp_copilot_gray_to_binary_0001	Code completion	0	3	0	3	0
cvdp_copilot_reverse_bits_0001	Description-to-RTL	3	3	3	3	0

Table 7: Per-task result summary across repeated runs.

References

- Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8, 2023. doi: 10.1109/ICCAD57390.2023.10323812.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RtlM: An open-source benchmark for design rtl generation with large language model. *arXiv preprint arXiv:2308.05345*, 2023.
- Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucek Khailany, and Haoxing Ren. Comprehensive verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on rtl design and verification. *arXiv preprint arXiv:2506.14074*, 2025.